

C++ – Smart Pointers

Adam HAMOUC

May 2026

1. Définition

Les **raw pointers** ont une sémantique floue : ownership incertain, risque de double-delete ou de fuite mémoire. Les **smart pointers** appliquent le principe RAI¹ et gèrent la durée de vie automatiquement.

Trois types : `unique_ptr`, `shared_ptr`, `weak_ptr`.

2. `std::unique_ptr`

Sémantique

Propriété **exclusive** : un seul owner à la fois. **Movable**, non copiable. Se détruit automatiquement à la sortie de scope.

Construction

```
// Toujours make_unique, jamais new direct
auto mesh = std::make_unique<Mesh>(vertices, indices);

// Forward si template tres generique
template<typename T, typename... Args>
auto makeAsset(Args&&... args) {
    return std::make_unique<T>(std::forward<Args>(args)...);
}
```

Utilisations typiques en engine

```
// Factory de composants : le caller devient owner
std::unique_ptr<RenderComponent> createRenderer(MeshID id);

// Lien parent-enfant dans la scene graph
struct SceneNode {
    std::vector<std::unique_ptr<SceneNode>> children;
};

// Pimpl Idiom (voir section 6)
class Renderer { std::unique_ptr<Impl> pImpl; };
```

¹Resource Acquisition Is Initialization : la ressource est libérée automatiquement à la destruction de l'objet.

`reset()` reprend l'ownership et détruit l'ancienne ressource.
Un `delete` custom est possible mais augmente la taille en mémoire (moins si `lambda`).

3. `std::shared_ptr`

Sémantique

Propriété **partagée** via compteur de référence. La ressource est détruite quand le compteur tombe à 0 (destruction déterministe, contrairement à un GC).

Construction

```
// make_shared = 1 seule allocation (objet + control block)
auto tex = std::make_shared<Texture>("albedo.png");

// new = 2 allocations, à éviter
std::shared_ptr<Texture> tex2(new Texture("albedo.png")); // bof
```

Utilisation typique en engine

```
// Ressource partagée entre plusieurs meshes
std::shared_ptr<Texture> albedo = assetManager.load("rock.png");
meshA->setAlbedo(albedo);
meshB->setAlbedo(albedo); // même texture, même alloc
```

Coûts : 2x plus grand qu'un raw pointer, incrément/décrément atomique du compteur.
`make_shared` inutilisable si `delete` custom complexe → fallback sur `new` dans ce cas seulement.

4. `std::weak_ptr`

Sémantique

Dérivé d'un `shared_ptr`. N'affecte **pas** le compteur de référence. Pointe sur la ressource sans en être owner. Permet de vérifier si la ressource est encore vivante avant d'y accéder.

```
std::weak_ptr<Texture> wpTex = albedo; // pas de +1 sur le compteur

// Accès sécurisé : lock() renvoie un shared_ptr ou nullptr si expired
if (auto spTex = wpTex.lock()) {
    spTex->bind(slot); // ok, la texture est encore vivante
}
```

Cas d'usage engine

```
// Pattern Observer : un listener ne doit pas retenir son subject
class LightObserver {
    std::weak_ptr<Light> observedLight; // pas d'ownership
}
```

```
};

// Cache de ressources : evite de recharger sans garder en vie
std::unordered_map<std::string, std::weak_ptr<Texture>> textureCache;
// Si tous les owners ont detruit la texture, weak_ptr l'indique
    proprement.
```

Evite deux anti-patterns : les cycles de `shared_ptr` (deadlock memoire) et la perte de reference des raw pointers. Meme cout memoire qu'un `shared_ptr`.

5. Pimpl Idiom

Problème

Sur les gros projets, modifier un header lourd (`vulkan.h`, `PhysicsBody.h...`) déclenche la recompilation de tous les fichiers qui l'incluent.

Solution

Cacher les membres privés dans une `struct Impl` déclarée dans le `.h` mais définie uniquement dans le `.cpp`.

```
// renderer.h      aucun include lourd
#pragma once
#include <memory>
class Renderer {
public:
    Renderer();
    ~Renderer();           // declare ici
    Renderer(Renderer&&);
    Renderer& operator=(Renderer&&);
    void draw(const Mesh& mesh);
private:
    struct Impl;           // type incomplet, juste declare
    std::unique_ptr<Impl> pImpl;
};
```

```
// renderer.cpp    tous les includes lourds ici
#include "renderer.h"
#include <vulkan/vulkan.h> // Vulkan reste dans le .cpp
#include <glm/glm.hpp>
#include "ShaderCache.h"

struct Renderer::Impl {
    VkDevice    device;
    ShaderCache shaderCache;
    glm::mat4    viewProj;
};

// Tout est defini apres Impl => type complet => OK
Renderer::Renderer() : pImpl(std::make_unique<Impl>()) {}
Renderer::~~Renderer() = default;
```

```

Renderer::Renderer(Renderer&&) = default;
Renderer& Renderer::operator=(Renderer&&) = default;

void Renderer::draw(const Mesh& mesh) {
    pImpl->shaderCache.bind(...); // acces interne uniquement
}

```

Trois regles unique_ptr + Pimpl :

- **Déclare** destructeur et move dans le .h, ne les définis pas.
- **Définis** tout dans le .cpp, après la définition complète de Impl.
- **Copie** non générée automatiquement → à écrire manuellement dans le .cpp si besoin.

Le contenu d'Impl est **strictement privé** : seules les méthodes de la classe peuvent accéder à pImpl->....

6. Pièges classiques

- Ne jamais construire un unique_ptr/shared_ptr depuis un raw pointer déjà géré ailleurs → double delete.
- Cycles de shared_ptr (A pointe sur B, B pointe sur A) → fuite mémoire, casser avec weak_ptr.
- Oublier de déclarer le destructeur avec Pimpl + unique_ptr → erreur cryptique à la compilation (delete sur type incomplet).
- make_shared inutilisable avec un deleter custom non trivial.
- Liens **parent**→**enfant** dans une scene graph : unique_ptr (propriété exclusive). Liens **enfant**→**parent** si nécessaire : weak_ptr (jamais shared_ptr, sinon cycle).

7. Coût et performance

	unique_ptr	shared_ptr	weak_ptr
Taille	= raw ptr	2× raw ptr	2× raw ptr
Overhead runtime	nul	atomique (ref count)	atomique (lock)
Deleter custom	augmente taille	non	—

8. Quand utiliser / éviter

8.1 unique_ptr

- Propriété exclusive : composants d'une scene graph, factory, Pimpl.

- Par défaut, substitut direct du raw pointer avec ownership clair.

8.2 `shared_ptr`

- Ressources partagées : textures, meshes, shaders chargés par un asset manager.
- À éviter si ownership exclusif suffisant → overhead inutile.

8.3 `weak_ptr`

- Observer, cache de ressources, lien enfant→parent dans une hiérarchie.
- Partout où l'on veut "voir" sans "posséder".

8.4 Éviter les smart pointers

- Paramètre de fonction sans transfert d'ownership → passer par référence ou raw pointer (pas d'ownership = pas de smart pointer).
- Chemins ultra-hot (boucle de rendu critique) si l'overhead atomique devient mesurable.